Method Expectations Replace Their Originals



People new to RSpec are often surprised at the behavior of expect on a partial double. The following code:

```
expect(some_existing_object).to receive(:a_message)
```

…doesn't just set up an expectation. It also *changes* the behavior of the existing object. Calls to some_existing_object.a_message will return nil and do nothing else. If you want to add a message expectation while retaining the original implementation, you'll need to use and_call_original.

Returning Multiple Values

You've already used and_return, in *Test Doubles: Mocks, Stubs, and Others*, on page 67. There, you set up your test double to return the same canned expense item each time it received the record message.

Sometimes, you need your stubbed method to do something more sophisticated than return the same value every time it's called. You might want to return one value for the first call, a different one for the second call, and so on.

For these cases, you can pass multiple values to and_return:

```
>> allow(random).to receive(:rand).and_return(0.1, 0.2, 0.3)
=> #<RSpec::Mocks::MessageExpectation #<Double "Random">.rand(any arguments)>
>> random.rand
=> 0.1
>> random.rand
=> 0.2
>> random.rand
=> 0.3
>> random.rand
=> 0.3
>> random.rand
=> 0.3
```

Here, we give three return values, and the rand method returns each one in sequence. After the third call, the method continues to return 0.3, the final value passed to and_return.

Yielding Multiple Values

Blocks are ubiquitous in Ruby, and sometimes your test doubles will need to stand in for an interface that uses blocks. The aptly named and_yield method will configure your double to yield values.

To specify a sequence of values to yield, chain together multiple calls to and_yield: